

Projeto de desenvolvimento Software

Relatório técnico - Hotel

Docente:

Fernando Pinho Marson

Nayuka Jasmin Malebo, 20220386

2026

Índice

1. Introdução

2. Arquitetura Tecnológica

3. Modelo de Dados

4. Backend - Arquitetura e Implementação

5. Frontend - Implementação

6. Regras de Negócio Implementadas

7. Sistema de Permissões e Acesso

8. Segurança

9. Performance e Otimização

10. Testes e Validação

Introdução

Objetivo do Projeto

Sistema web completo para gestão integral de um hotel, desenvolvido como projeto acadêmico. O sistema permite a gestão de reservas, quartos, hóspedes, pagamentos e relatórios, com diferentes níveis de acesso para clientes, recepcionistas e gestores.

Escopo

- Área do Cliente: pesquisa, reserva e gestão de reservas
- Área da Gerência: administração completa do hotel (quartos, reservas, pagamentos, relatórios)
- Sistema de autenticação e autorização baseado em roles
- API RESTful completa
- Interface web responsiva

Tecnologias usadas

Frontend

- **Next.js 14.0.4**: Framework React com App Router
- **React 18.2.0**: Biblioteca UI
- **TypeScript 5.3.3**: Tipagem estática
- **Tailwind CSS 3.4.0**: Framework CSS utility-first
- **Zustand 4.4.7**: Gestão de estado global
- **Axios 1.6.2**: Cliente HTTP
- **React Hook Form 7.49.2**: Gestão de formulários
- **date-fns 3.0.6**: Manipulação de datas
- **lucide-react 0.303.0**: Ícones

Backend

- **Node.js**: Runtime JavaScript
- **Express 4.18.2**: Framework web
- **TypeScript 5.3.3**: Tipagem estática
- **Prisma 5.7.1**: ORM type-safe
- **MySQL**: Base de dados relacional
- **JWT**: Autenticação
- **bcrypt 5.1.1**: Hash de passwords
- **express-validator 7.0.1**: Validação de dados
- **cors 2.8.5**: Cross-Origin Resource Sharing

Base de Dados

Modelo de Dados

O sistema utiliza MySQL como base de dados, com schema definido através do Prisma ORM. O modelo de dados inclui as seguintes entidades principais:

Utilizador

- **Campos**:
 - ``id``: Identificador único (CUID)
 - ``email``: Email único
 - ``password``: Hash da password (bcrypt)
 - ``tipo``: Enum (CLIENTE, RECECIONISTA, GESTOR)
 - ``nome``: Nome completo
 - ``ativo``: Estado ativo/inativo
- **Relações**: Reservas, Pagamentos, LogsAuditoria

TipoQuarto

- Campos:

- `id`: Identificador único
- `nome`: Nome do tipo (único)
- `descricao`: Descrição opcional
- `valorBaseDiaria`: Preço base por noite
- `capacidadeBase`: Número de hóspedes base
- `suplementoHospedeExtra`: Valor por hóspede extra
- `custoPequenoAlmoco`: Custo do pequeno-almoço por hóspede
- `ativo`: Estado ativo/inativo

- Relações: Quartos, Reservas

Quarto

- Campos:

- `id`: Identificador único
- `numero`: Número do quarto (único)
- `tipoQuartoId`: Referência ao tipo de quarto
- `estado`: Enum (LIVRE, OCUPADO, MANUTENCAO)

- Relações: TipoQuarto, ReservaQuarto

Hospede

- Propósito: Informação dos hóspedes

- Campos:

- `id`: Identificador único
- `nome`: Nome completo
- `tipoDocumento`: Enum (CARTAO_CIDADAO, PASSAPORTE, OUTRO)
- `numeroDocumento`: Número do documento (único com tipoDocumento)
- `nif`: NIF opcional para faturação
- `ativo`: Estado ativo/inativo

- Relações: ReservaHospede

Reserva

- Campos:

- `id`: Identificador único
- `utilizadorId`: Cliente que fez a reserva
- `tipoQuartoId`: Tipo de quarto reservado
- `dataInicio`: Data de check-in
- `dataFim`: Data de check-out
- `quantidadeQuartos`: Número de quartos
- `hospedesPorQuarto`: Número de hóspedes por quarto
- `incluirPequenoAlmoco`: Boolean
- `estado`: Enum (ATIVA, CANCELADA, CONCLUIDA)
- `totalCalculado`: Total calculado da reserva
- `checkInEfetuado`: Boolean
- `checkOutEfetuado`: Boolean

- **Relações:** Utilizador, TipoQuarto, ReservaQuarto, ReservaHospede, Pagamento

ReservaQuarto

- `id`: Identificador único
- `reservaId`: Referência à reserva
- `quartoId`: Referência ao quarto
- **Constraint:** Unique (reservaId, quartoId)

ReservaHospede (Tabela de Junção)

- **Propósito:** Relação many-to-many entre Reserva e Hospede

- Campos:

- `id`: Identificador único
- `reservaId`: Referência à reserva
- `hospedeId`: Referência ao hóspede
- **Constraint:** Unique (reservaId, hospedeId)

Pagamento

- **Propósito:** Registro de pagamentos
- **Campos:**
 - `id`: Identificador único
 - `reservaId`: Referência à reserva
 - `utilizadorId`: Operador que registou o pagamento
 - `montante`: Valor pago
 - `data`: Data do pagamento
 - `tipo`: Enum (PARCIAL, TOTAL)
 - `observacoes`: Observações opcionais
- **Relações:** Reserva, Utilizador

LogAuditoria

- `id`: Identificador único
- `acao`: Tipo de ação (string)
- `entidade`: Entidade afetada (string)
- `entidadeId`: ID da entidade afetada
- `utilizadorId`: Utilizador que executou a ação
- `detalhes`: Detalhes adicionais (text)
- `ip`: Endereço IP
- `userAgent`: User agent do browser
- `createdAt`: Data/hora da ação
- **Índices:** acao, entidade, createdAt

Enums

- **TipoDocumento:** CARTAO_CIDADAOS, PASSAPORTE, OUTRO
- **TipoUtilizador:** CLIENTE, RECECIONISTA, GESTOR
- **EstadoQuarto:** LIVRE, OCUPADO, MANUTENCAO
- **EstadoReserva:** ATIVA, CANCELADA, CONCLUIDA
- **TipoPagamento:** PARCIAL, TOTAL

Migrations

O sistema utiliza Prisma Migrate para gestão de versões do schema. As migrations são versionadas e aplicadas automaticamente através do comando ``prisma migrate dev``.

Backend

Servidor Express

O servidor Express está configurado em ``backend/src/server.ts``:

- **Porta:** 3001 (configurável via ``PORT`` env)
- **CORS:** Configurado para permitir requisições do frontend
- **Middleware:** JSON parser, URL encoded parser
- **Rotas:**
 - ``/api/auth``: Autenticação
 - ``/api/cliente``: Operações do cliente
 - ``/api/gerencia``: Operações da gerência
 - ``/api/relatorios``: Relatórios
- **Error Handling:** Middleware global para tratamento de erros
- **Health Check:** Endpoint ``/health`` para verificação de status

Autenticação e Autorização

Middleware de Autenticação

Localizado em ``backend/src/middleware/auth.middleware.ts``:

- **authenticate:** Verifica token JWT e valida utilizador
- **requireRole:** Middleware genérico para verificação de roles
- **requireGestor:** Apenas Gestor
- **requireRececionista:** Rececionista ou Gestor
- **requireAdmin:** Rececionista ou Gestor

JWT (JSON Web Tokens)

- **Algoritmo:** HS256
- **Payload:** `{ userId, userType }`
- **Secret:** Configurado via `JWT\_SECRET` env

Hash de Passwords

- **Biblioteca:** bcrypt
- **Rounds:** 10
- **Armazenamento:** Hash no campo `password` da tabela `utilizadores`

Rotas da API

Autenticação (`/api/auth`)

POST /api/auth/registro

- Registo de novos utilizadores (sempre como CLIENTE)
- Validação: email único, password mínimo 6 caracteres
- Retorna: token JWT e dados do utilizador

POST /api/auth/login

- Autenticação de utilizadores existentes
- Validação: email e password
- Retorna: token JWT e dados do utilizador

GET /api/auth/perfil

- Obtém perfil do utilizador autenticado
- Requer: autenticação

Cliente (`/api/cliente`)

Todas as rotas requerem autenticação.

GET /api/cliente/tipos-quarto

- Lista tipos de quarto disponíveis
- Retorna: lista de tipos de quarto ativos

POST /api/cliente/verificar-disponibilidade

- Verifica disponibilidade de quartos para um período
- Parâmetros: tipoQuartoId, dataInicio, dataFim, quantidadeQuartos
- Retorna: disponibilidade, quartos disponíveis, preço estimado

POST /api/cliente/reservas

- Cria nova reserva
- Validações:
 - Datas válidas (início < fim, não no passado)
 - Disponibilidade de quartos
 - Capacidade de hóspedes
- Atribui quartos automaticamente
- Calcula total automaticamente
- Cria/atualiza hóspedes
- Retorna: reserva criada

GET /api/cliente/reservas

- Lista reservas do utilizador autenticado
- Retorna: lista de reservas com detalhes

GET /api/cliente/reservas/:id

- Obtém detalhes de uma reserva específica
- Validação: reserva pertence ao utilizador

POST /api/cliente/reservas/:id/cancelar

- Cancela uma reserva
- Validação: regra das 24 horas
- Libera quartos associados
- Retorna: reserva cancelada

PUT /api/cliente/reservas/:id

- Edita uma reserva existente
- Validações:
 - Regra das 24 horas
 - Disponibilidade para novas datas/quantidade
 - Reserva deve estar ATIVA
- Recalcula total
- Reatribui quartos se necessário
- Retorna: reserva atualizada

Gerência (/api/gerencia')

Todas as rotas requerem autenticação e permissões de admin (Rececionista ou Gestor).

Tipos de Quarto

GET /api/gerencia/tipos-quarto

- Lista todos os tipos de quarto
- Acesso: Rececionista e Gestor

POST /api/gerencia/tipos-quarto

- Cria novo tipo de quarto
- Acesso: Apenas Gestor
- Validação: nome único

PUT /api/gerencia/tipos-quarto/:id

- Atualiza tipo de quarto
- Acesso: Apenas Gestor

Quartos

GET /api/gerencia/quartos

- Lista todos os quartos
- Inclui: tipo de quarto, estado, reservas

POST /api/gerencia/quartos

- Cria novo quarto
- Validação: número único

PUT /api/gerencia/quartos/:id

- Atualiza quarto (principalmente estado)

Hóspedes

GET /api/gerencia/hospedes

- Lista todos os hóspedes
- Filtros: nome, documento, NIF

POST /api/gerencia/hospedes

- Cria novo hóspede
- Validação: tipoDocumento + numeroDocumento único

PUT /api/gerencia/hospedes/:id

- Atualiza hóspede

Reservas

GET /api/gerencia/reservas

- Lista todas as reservas
- Filtros: estado, data, cliente
- Inclui: cliente, tipo de quarto, quartos, hóspedes, pagamentos

GET /api/gerencia/reservas/:id

- Obtém detalhes completos de uma reserva

POST /api/gerencia/reservas/:id/checkin

- Efetua check-in
- Atualiza estado dos quartos para OCUPADO
- Marca checkInEfetuado = true

POST /api/gerencia/reservas/:id/checkout

- Efetua check-out
- Atualiza estado dos quartos para LIVRE
- Marca checkOutEfetuado = true
- Atualiza estado da reserva para CONCLUIDA

POST /api/gerencia/reservas/:id/cancelar

- Cancela reserva (admin)
- Libera quartos

Pagamentos

GET /api/gerencia/pagamentos

- Lista todos os pagamentos
- Inclui: reserva, cliente, operador
- Filtros: data, tipo, reserva

POST /api/gerencia/pagamentos

- Registra novo pagamento
- Validação: montante > 0, reserva existe
- Calcula saldo pendente

GET /api/gerencia/pagamentos/:id/comprovativo

- Gera comprovativo de pagamento
- Retorna: dados do pagamento, reserva, saldo

Utilizadores

GET /api/gerencia/utilizadores

- Lista todos os utilizadores
- Acesso: Apenas Gestor
- Não retorna passwords

POST /api/gerencia/usuarios

- Cria novo utilizador (Rececionista ou Gestor)
- Acesso: Apenas Gestor
- Validação: email único

Relatórios ('/api/relatorios')

Todas as rotas requerem autenticação e permissões de admin.

GET /api/relatorios/ocupacao-diaria

- Ocupação diária para uma data específica
- Parâmetros: data (query)
- Retorna: quartos ocupados, taxa de ocupação

GET /api/relatorios/ocupacao-mensal

- Ocupação mensal para um mês/ano
- Parâmetros: mes, ano (query)
- Retorna: ocupação por dia do mês

GET /api/relatorios/reservas-periodo

- Reservas em um período
- Parâmetros: dataInicio, dataFim (query)
- Retorna: lista de reservas com detalhes

GET /api/relatorios/receita-periodo

- Receita em um período
- Parâmetros: dataInicio, dataFim (query)
- Retorna: receita total, por tipo de quarto, por dia

GET /api/relatorios/historico-hospedes

- Histórico de hóspedes
- Filtros: nome, documento
- Retorna: hóspedes com suas reservas

GET /api/relatorios/logs-auditoria

- Logs de auditoria
- Filtros: acao, entidade, data
- Retorna: lista de logs com detalhes

Cálculo de Total da Reserva

Função: `calcularTotalReserva` em `backend/src/utis/reserva.util.ts`

Fórmula:

```

Número de noites = dataFim - dataInicio

Valor base por quarto por noite = tipoQuarto.valorBaseDiaria

Se hospedesPorQuarto > capacidadeBase:

Suplemento = (hospedesPorQuarto - capacidadeBase) × suplementoHospedeExtra

Valor por quarto por noite += Suplemento

Se incluirPequenoAlmoco:

Valor por quarto por noite += hospedesPorQuarto × custoPequenoAlmoco

Total = Valor por quarto por noite × Número de noites × Quantidade de quartos

### **Verificação de Disponibilidade**

Função: `verificarDisponibilidade` em `backend/src/utis/[reserva.util.ts](#)`

#### **Lógica:**

1. Busca quartos do tipo solicitado com estado LIVRE
2. Verifica reservas ativas que se sobrepõem ao período
3. Conta quartos ocupados no período
4. Calcula quartos disponíveis = quartos livres - quartos ocupados
5. Compara com quantidade solicitada

### **Atribuição de Quartos**

Função: `atribuirQuartos` em `backend/src/utis/reserva.util.ts`

**Lógica:**

1. Busca quartos livres do tipo
2. Filtra quartos não ocupados no período
3. Seleciona os primeiros N quartos (onde N = quantidadeQuartos)
4. Retorna IDs dos quartos selecionados

**Regra das 24 Horas**

Função: `podeEditarCancelar` em `backend/src/utis/reserva.util.ts`

**Lógica:**

- Calcula diferença entre dataInicio da reserva e agora
- Se diferença > 24 horas: permite editar/cancelar
- Se diferença  $\leq$  24 horas: bloqueia edição/cancelamento

**Logs de Auditoria**

Sistema completo de logs de auditoria implementado em `backend/src/utis/logger.util.ts`:

**- Ações registradas:**

- Registro\_utilizador
- Login
- Criar\_reserva
- Editar\_reserva
- Cancelar\_reserva
- Check\_in
- Check\_out
- Registrar\_pagamento
- Criar\_quarto
- Atualizar\_quarto
- Criar\_tipo\_quarto
- Atualizar\_tipo\_quarto
- Criar\_hospede
- Atualizar\_hospede
- Criar\_utilizador



**- Informações capturadas:**

- Ação executada
- Entidade afetada
- ID da entidade
- Utilizador que executou
- Detalhes adicionais
- IP do cliente
- User Agent

## **Frontend**

### **Páginas e Funcionalidades**

#### **Autenticação**

##### **Página de Login (`/login`):**

- Formulário de email e password
- Validação de campos
- Redirecionamento baseado em tipo de utilizador
- Tratamento de erros

##### **Página de Registo (`/registro`):**

- Formulário: nome, email, password, confirmPassword
- Validação: passwords coincidem, mínimo 6 caracteres
- Registo automático como CLIENTE

#### **Área do Cliente**

##### **Página Principal (`/cliente`):**

- Dashboard com opções de navegação
- Links para: pesquisar quartos, minhas reservas

##### **Pesquisar e Reservar (`/cliente/reservar`):**

- Seleção de tipo de quarto
- Seleção de datas (dataInicio, dataFim)

- Seleção de quantidade de quartos
- Número de hóspedes por quarto
- Opção de incluir pequeno-almoço
- Cálculo em tempo real do total estimado
- Verificação de disponibilidade
- Formulário dinâmico de hóspedes
- Criação de reserva

#### **Minhas Reservas** (`/cliente/reservas`):

- Lista de reservas organizadas por:
  - Reservas Futuras
  - Reservas em Curso
  - Reservas Passadas
- Informação sobre regra das 24 horas
- Botão para editar reservas futuras (se permitido)
- Botão para cancelar reservas futuras (se permitido)
- Visualização de detalhes completos

#### **Editar Reserva** (`/cliente/reservas/[id]/editar`):

- Formulário pré-preenchido com dados atuais
- Possibilidade de alterar: datas, quantidade de quartos, hóspedes, pequeno-almoço
- Verificação de disponibilidade para novas datas
- Cálculo de novo total
- Validação da regra das 24 horas

### **Área da Gerência**

#### **Dashboard** (`/gerencia`):

- Menu de navegação para todas as funcionalidades
- Acesso baseado em permissões

#### **Gestão de Quartos** (`/gerencia/quartos`):

- Lista de todos os quartos
- Filtros por tipo e estado
- Criar novo quarto

- Alterar estado do quarto
- Visualização de reservas associadas

#### **Tipos de Quarto** (`/gerencia/tipos-quarto`):

- Lista de tipos de quarto
- Criar tipo (apenas Gestor)
- Editar tipo (apenas Gestor)
- Visualização de preços e capacidades

#### **Hóspedes** (`/gerencia/hospedes`):

- Lista de hóspedes
- Pesquisa por nome, documento, NIF
- Criar novo hóspede
- Editar hóspede
- Histórico de reservas do hóspede

#### **Reservas** (`/gerencia/reservas`):

- Lista de todas as reservas
- Filtros por estado, data, cliente
- Visualização de detalhes completos
- Check-in / Check-out
- Cancelar reserva

#### **Pagamentos** (`/gerencia/pagamentos`):

- Lista de pagamentos
- Registrar novo pagamento
- Seleção de reserva ativa
- Cálculo de saldo pendente
- Gerar comprovativo (HTML imprimível)

#### **Utilizadores** (`/gerencia/utilizadores`):

- Lista de utilizadores (apenas Gestor)
- Criar utilizador (Rececionista ou Gestor)
- Visualização de permissões

### **Relatórios** (`/gerencia/relatorios`):

- Ocupação diária
- Ocupação mensal
- Reservas por período
- Receita por período
- Histórico de hóspedes
- Logs de auditoria
- Visualização formatada (não JSON bruto)

### **Validação e Tratamento de Erros**

- **Validação de Formulários:** React Hook Form
- **Validação de API:** Express Validator (backend)
- **Tratamento de Erros:** Try-catch em todas as chamadas API
- **Mensagens de Erro:** Exibidas em componentes de alerta
- **Loading States:** Indicadores de carregamento durante requisições

## **Regras de Negócio Implementadas**

### **Validações de Reserva**

#### **Datas:**

- Data de início deve ser anterior à data de fim
- Não permite reservas com datas no passado
- Verificação de disponibilidade para o período

#### **Capacidade:**

- Número de hóspedes por quarto deve ser  $\geq 1$
- Máximo: capacidadeBase + 2 hóspedes extras

**Disponibilidade:**

- Verifica quartos livres do tipo solicitado
- Verifica sobreposição com reservas ativas
- Previne overbooking

**Regra das 24 Horas:**

- Edição/cancelamento permitido apenas se  $\text{dataInicio} > \text{agora} + 24\text{h}$
- Aplicada tanto no backend quanto no frontend

**Cálculo de Preços**

1. **Diária Base:**  $\text{valorBaseDiaria} \times \text{número de noites} \times \text{quantidade de quartos}$
2. **Suplemento Hóspede Extra:**  $(\text{hóspedesExtras} \times \text{suplementoHospedeExtra}) \times \text{noites} \times \text{quartos}$
3. **Pequeno-Almoço:**  $(\text{hóspedesPorQuarto} \times \text{custoPequenoAlmoco}) \times \text{noites} \times \text{quartos}$
4. **Total:** Soma de todos os componentes, arredondado para 2 casas decimais

**Gestão de Quartos****Estados:**

- LIVRE: Disponível para reserva
- OCUPADO: Atualmente ocupado
- MANUTENCAO: Indisponível para manutenção

**Atribuição Automática:**

- Quartos são atribuídos automaticamente na criação da reserva
- Seleciona quartos livres não ocupados no período
- Atualiza estado para OCUPADO no check-in

**Liberação:**

- Quartos são liberados no cancelamento ou check-out
- Estado atualizado para LIVRE

## **Pagamentos**

### **Tipos:**

- PARCIAL: Pagamento parcial da reserva
- TOTAL: Pagamento total da reserva

### **Cálculo de Saldo:**

- Total pago = soma de todos os pagamentos da reserva
- Saldo pendente = totalCalculado - totalPago

### **Comprovativo:**

- Geração de comprovativo HTML imprimível
- Inclui: dados do pagamento, reserva, cliente, saldo

## **Permissões e Acesso**

### **CLIENTE:**

- Ver tipos de quarto
- Criar/editar/cancelar próprias reservas
- Ver próprias reservas

### **RECECIONISTA:**

- Todas as permissões de visualização
- Criar/editar/cancelar reservas (de qualquer cliente)
- Check-in/Check-out
- Registar pagamentos
- Ver relatórios
- Ver tipos de quarto (mas não criar/editar)

### **GESTOR:**

- Todas as permissões do Rececionista
- Criar/editar tipos de quarto
- Criar/editar utilizadores
- Acesso completo a todos os relatórios

# Segurança

## Autenticação

- **JWT Tokens:** Tokens assinados com secret
- **Expiração:** Tokens expiram após 7 dias
- **Validação:** Token validado em cada requisição protegida
- **Armazenamento:** Token no localStorage (frontend)

## Autorização

- **Role-Based Access Control (RBAC):** Baseado em tipos de utilizador
- **Middleware de Autorização:** Verificação de roles em rotas protegidas
- **Validação de Propriedade:** Clientes só acedem às próprias reservas

## Validação de Dados

- **Backend:** Express Validator para validação de entrada
- **Frontend:** React Hook Form para validação de formulários
- **Sanitização:** Validação de tipos, formatos, ranges

## Passwords

- **Hash:** bcrypt com 10 rounds
- **Armazenamento:** Apenas hash armazenado, nunca password em texto
- **Validação:** Comparação com hash na autenticação

## CORS

- **Configuração:** CORS configurado para permitir apenas origem do frontend
- **Credentials:** Suporte a credenciais habilitado

## Logs de Auditoria

- **Rastreabilidade:** Todas as ações críticas são registadas
- **Informações:** IP, User Agent, utilizador, timestamp
- **Consulta:** Logs disponíveis para gestores

## Performance e Otimizações

### Base de Dados

- **Índices:** Índices em campos frequentemente consultados (logs, reservas)
- **Queries Otimizadas:** Uso de includes do Prisma para evitar N+1 queries
- **Cascading Deletes:** Configurado para manter integridade

### Frontend

- **Next.js App Router:** Renderização otimizada
- **Lazy Loading:** Componentes carregados sob demanda
- **Cálculo em Tempo Real:** Total estimado calculado no cliente

### API

- **Validação Rápida:** Validações feitas antes de queries pesadas
- **Error Handling:** Tratamento eficiente de erros

## Testes e Validação

### Dados de Teste

Script de seed (`backend/prisma/seed.ts`) cria:

- Tipos de quarto (Standard, Deluxe, Suite, Solteiro)
- Quartos de cada tipo
- Utilizadores de teste (Cliente, Rececionista, Gestor)
- Reservas de exemplo



